

Skill-selection testsets: SkillsBench + Terminal-Bench

Comprehensive design + results report | [procmem2skills/testsets/](#) | 2026-05-05

Executive summary

Two complementary skill-selection testsets, both evaluated under a harbor-mimic prompt-and-parse harness with no agent execution. SkillsBench (88 tasks, native multi-GT) is the in-domain reference; Terminal-Bench validated (62 of 241 tasks, Opus-judge multi-GT) is the OOD complement. Together they produce a dataset-invariant selection-collapse curve, plus a cross-dataset failure mechanism: under hard distractors, the LLM picks within the embedding neighbourhood of the task but fails to discriminate the correct skill - and on TB it drops below a no-LLM embedding-argmax baseline.

One-line claim

Even when the correct skill is among the top-5 most similar candidates, LLMs under hard distractors select the correct skill only 41.9 % of the time - substantially below a simple embedding-argmax baseline (53.2 %). Errors concentrate within the nearest-neighbour region of the embedding space, indicating a failure of fine-grained discrimination rather than retrieval.

Artifacts in this commit

testsets/	
data/	
skillsbench_tasks.jsonl	88 tasks, native GT (1-7 GT/task)
terminal_bench_tasks.jsonl	241 raw tasks (no GT)
terminal_bench_topk.jsonl	FAISS top-5 candidates per TB task
terminal_bench_validated.jsonl	62 validated tasks, Opus-judge GT
figures/	
collapse_analysis.pdf	Figure 1 (rank/gap/oracle)
terminalbench-testset.pdf	TB-only construction summary
testsets-overview.pdf	this document
skill_selection_eval/	
judge_tb.py + aggregate_judge.py	Opus-judge pipeline (TB GT)
run.py + run_tb.py	harbor-mimic prompt-and-parse
pool_builder.py + tb_pool_builder.py	size/noise pool variants
prompt.py + parser.py	harbor-format I/O
RESULTS.md	full methodology + caveats

1. Design rationale

1.1 Two datasets, two roles

The two testsets are not redundant. They occupy different points in the 'is the correct skill in the candidate pool?' axis, which is the variable the paper needs to range over:

SkillsBench - in-domain reference

- GT source: native, authored alongside the SkillsBench corpus.
- Domain: skills-by-design tasks. The ClawHub corpus is in-distribution.
- Role: gives a clean upper-bound on selection accuracy when the testset and corpus are co-designed.

Terminal-Bench validated - OOD complement

- GT source: Opus-judge YES verdict on top-5 FAISS retrievals (1205 pairs -> 98 YES).
- Domain: real shell-agent tasks. ClawHub coverage is partial (74 % drop rate at GT construction).
- Role: stress-tests selection on tasks not designed around the corpus.

1.2 Methodology principle: retrieval isolation

Throughout, agent execution is NOT used as a ground-truth signal. The retrieval-isolation guardrail (`feedback\_retrieval\_isolation.md`) was set early to prevent the circular pattern of 'skill helps the agent solve the task -> label as GT'. For SkillsBench this is automatic - GT is authored. For TB, the natural temptation was incremental harbor injection (the original todo (2)); we declined that path and substituted Opus LLM-as-judge instead.

Why a separate judge model (Opus) rather than the eval target (Sonnet): keeps the GT-construction and evaluation models distinct -> no train/test leakage in capability terms.

1.3 Why harbor-mimic, not harbor-actual

Harbor's `terminus-2-skills` agent (a) calls models via LiteLLM/API only - no Max-plan support; (b) requires docker + per-task verifiers; (c) charges per-task API of ~\$1-3 on Sonnet, scaling to \$700-\$1.7 K for the GT-construction loop alone. Pure forward-pass replication of harbor's `` XML prompt + `

1.4 Pool design - size x noise factorial

Both datasets are evaluated on the same factorial:

pool_size	in {5, 50, 500}
noise_mode	in {topk_only, random, easy, hard, none}
topk_only	: K FAISS retrievals only - no distractors
random	: K FAISS retrievals + uniformly random ClawHub fillers
easy	: K FAISS retrievals + farthest-from-task fillers (cosine bottom)
hard	: K FAISS retrievals + nearest-non-GT-neighbour fillers
none	: no GT in pool - measures refusal calibration

`hard` is the adversarial condition the paper centres on: distractors are pulled from the GT's own embedding neighbourhood, so the model cannot dispatch them by similarity alone.

2. Pipelines

2.1 SkillsBench (in-domain)

```
skillsbench_tasks.jsonl  (88 tasks, native multi-GT)
    |
    v
pool_builder.py  ----  build_pool(size, noise, seed)
                    [GT injected, distractors per noise mode]
    |
    v
prompt.py       <available_skills> XML block (harbor-format)
    |
    v
run.py          claude -p Max plan -> response
    |
    v
parser.py       <tool_call name="skill"><name>X</name></tool_call> regex
    |
    v
aggregate.py    Hit@1 / Recall / MRR / refusal per (size, noise, seed)
```

2.2 Terminal-Bench (OOD)

```
terminal_bench_tasks.jsonl  (241 raw tasks, no GT)
    |
    v
retrieve_topk.py  Qwen3-Embedding-0.6B + FAISS over 44,787 ClawHub skills
                  top-5 per task -> terminal_bench_topk.jsonl
    |
    v
judge_tb.py      Opus 4.7 per (task, candidate): YES/NO + reason
                  1205 calls in ~25 min via claude -p Max plan, $0
    |
    v
aggregate_judge.py  Drop tasks where all 5 are NO
                   62 tasks kept (40 single-GT, 22 multi-GT)
    |
    v
terminal_bench_validated.jsonl  <-- the published testset
    |
    v
tb_pool_builder.py + run_tb.py  same harbor-mimic harness as SB
    |
    v
aggregate_tb / analyze_collapse  metrics + Figure 1
```

2.3 Output schemas

SkillsBench task record

```
{"task_id": "3d-scan-calc", "task_description": "...", "gt_skills": ["mesh-analysis"]}
```

TB validated task record (richer - judge metadata preserved)

```
{"task_id": "accelerate-maximal-square",
 "task_description": "...",
 "gt_skills": [{"skill_id": "skill_043613", "slug": "ivangdavila/numpy",
                  "name": "NumPy", "rank": 1,
                  "reason": "Task requires vectorising a NumPy DP loop..."}],
 "n_yes": 1, "n_judged": 5}
```

3. Todo completion (combined matrix)

Original spec, paraphrased:

- (1) SkillsBench: build a testset using the native ground-truth skills.
- (2) Terminal-Bench: no native GT - derive GT by per-skill helpfulness signal; drop tasks with no helpful skill; push metadata to GitHub.
- (3) Build a test script that mimics harbor's task delivery: feed task description, require the agent to emit selected skills, parse output.
- (4) Vary distractors: pool sizes {5, 50, 500} x noise {random, hard, easy}.
- (5) Analyse model outputs, compute pick accuracy, log results.

Status matrix

Spec point	SkillsBench	Terminal-Bench	Note
(1) Build testset from native GT	DONE	N/A	SB has authored multi-GT (1-7 skills/task); used directly. TB has no native GT.
(2a) Top-K retrieval (TB)	N/A	DONE	TB only: Qwen3-Embedding + FAISS top-5 over 44,787 ClawHub skills.
(2b) Per-skill helpfulness signal	N/A	SUBSTITUTED	Originally: incremental harbor injection. Replaced by Opus LLM-as-judge after harbor ruled out (no Max-plan, \$700-1.7K cost). 1205 pairs judged in 25 min, \$0.
(2c) Drop tasks with no GT	N/A	DONE	179 / 241 dropped (74.3 %); 62 kept. Sets a retrievability-conditioned subset.
(2d) Push metadata to GitHub	DONE	DONE	Single commit covers both testsets. (Awaiting push-target decision: fork / upstream branch / upstream master.)
(3) Harbor-mimic test script	DONE	DONE	prompt.py replicates <available_skills> XML; parser.py uses harbor's tool_call regex; run.py / run_tb.py drive Max-plan claude -p with rate-limit + resume.
(4) Pool size x noise grid	DONE	DONE	Both datasets use the same factorial: pool sizes {5, 50, 500} x noise modes {topk_only, random, hard, easy, none}.
(5) Pick-accuracy analysis	DONE	DONE	SB: 1068 trials, 0 errors, full Hit@1 / Recall / MRR table. TB: 930 trials, 0 errors, same metrics + cross-dataset comparison + Figure 1 embedding-neighbourhood diagnostics.

4. Headline results

4.1 Tasks-at-a-glance

SB tasks (kept)	88 (drop: 1 with empty GT)
SB GT-count distribution	1: 26 2: 20 3: 19 4: 10 5: 8 6: 4 7: 1
SB multi-GT (≥ 2)	62 of 88 (70 %)
TB tasks raw	241
TB validated (kept)	62 of 241 (25.7 %); 179 dropped
TB GT-count distribution	1: 40 2: 12 3: 7 4: 2 5: 1
TB multi-GT (≥ 2)	22 of 62 (35 %)

4.2 Validated-grid metrics - side-by-side

Both grids run with claude-sonnet-4-5 via Max-plan claude -p, 0 errors.

pool sz	noise	SB Hit@1 (n=89)	TB Hit@1 (n=62)	SB refusal	TB refusal
5	topk_only	-	74.2 %	-	16.1 %
5	random	80.9 %	74.2 %	18.0 %	12.9 %
5	hard	59.6 %	69.4 %	10.1 %	17.7 %
5	easy	85.4 %	75.8 %	14.6 %	17.7 %
5	none	-	-	97.8 %	100.0 %
50	topk_only	-	80.6 %	-	11.3 %
50	random	83.2 %	75.8 %	15.7 %	12.9 %
50	hard	55.1 %	54.8 %	12.4 %	19.4 %
50	easy	80.9 %	75.8 %	18.0 %	17.7 %
50	none	-	-	95.5 %	96.8 %
500	topk_only	-	79.0 %	-	17.7 %
500	random	76.4 %	58.1 %	20.2 %	30.6 %
500	hard	41.6 %	41.9 %	12.4 %	27.4 %
500	easy	71.9 %	48.4 %	28.1 %	40.3 %
500	none	-	-	82.0 %	83.9 %

4.3 Cross-dataset findings

(i) Selection collapse is dataset-invariant

Hard-noise Hit@1 collapse: SB 59.6 % $\rightarrow$ 41.6 % (sz 5 $\rightarrow$ 500). TB 69.4 % $\rightarrow$ 41.9 %. Same shape, near-identical floor. Cross-dataset confirmation of a structural failure mode, not a dataset artifact.

(ii) Refusal-when-no-GT degrades with pool size on both

'none' refusal: SB 97.8 / 95.5 / 82.0 vs TB 100.0 / 96.8 / 83.9. Identical signature. 16-18 % hallucination rate at sz=500 across both datasets is the cleanest cross-dataset safety claim.

(iii) topk_only is the stability ceiling

On TB, topk_only at sz=50 (80.6 %) and sz=500 (79.0 %) nearly equals sz=5 (74.2 %). When the K FAISS retrievals are presented without surrounding noise, large pool size is harmless. The failure mode is distractor density in the embedding neighbourhood, not pool size per se.

5. The mechanism (Figure 1)

Three diagnostics from the TB validated grid (figures/collapse_analysis.pdf), no new experiments - just rank/similarity post-hoc on existing 590 informative trials:

5.1 Wrong picks live near the task in embedding space

rank <= 1 (corpus argmax)	24.7 % of wrong picks (uniform = 0.002 %, x 11,071)
rank <= 5	61.8 % of wrong picks (uniform = 0.011 %, x 5,535)
rank <= 10	71.9 % of wrong picks (uniform = 0.022 %, x 3,221)
rank <= 50	91.0 % of wrong picks
median rank	4 of 44,787

5.2 No preference for GT in similarity space

median delta (correct picks)	0.000 (model picks the highest-sim GT)
median delta (wrong picks)	+0.022 (only marginally further than GT)
 delta < 0.05 (wrong picks)	49.4 % (basically tied)
delta < 0 (wrong picks)	28.1 % (model picked something MORE task-similar than GT)

5.3 LLM versus no-LLM baselines

On TB, GT is selected from the top-5 retrievals by construction, so embedding-only argmax is a fair ceiling baseline:

random-from-top-5 (no LLM)	31.3 %
LLM @ sz=500 hard	41.9 % (in red zone, below argmax)
LLM @ sz=500 easy	48.4 % (in red zone)
embedding-argmax oracle (no LLM, deterministic)	53.2 %
LLM @ sz=50 topk_only (clean condition)	80.6 %

The LLM beats random-from-top-5 (it does use embedding signal) but falls below a deterministic argmax under hard distractors (it is not exploiting the signal as well as a 1024-d cosine would).

5.4 Agreement: model vs embedding argmax

P(model = emb_argmax) overall	44.4 %
P(model = emb_argmax correct)	47.9 %
P(model = emb_argmax wrong)	24.7 %

The model is not a copy of an embedding selector - when wrong, it disagrees with emb_argmax 75 % of the time. The errors are model-specific, but they remain inside the embedding neighbourhood.

6. Caveats

6.1 Shared

- Single seed (0); no variance bars yet. Re-run {0,1,2} for paper-grade CIs (~3x compute, still \$0 on Max plan).
- Pool builder seeds are stable so resume / cross-run comparisons are deterministic.
- Eval model is fixed at claude-sonnet-4-5. Cross-model robustness (Opus, GPT-5, Gemini) not tested.

6.2 SkillsBench

- `max_gt = 1` in the published 1068-trial grid: multi-GT tasks are subsampled to a single GT per trial. Multi-GT recall is not yet measured end-to-end.
- 25.5 % of SB GT skills have a near-duplicate alias in ClawHub (cosine ≥ 0.9). The pool builder excludes the aliased corpus ID from the noise, so 'misses' under the alias boundary represent the model flipping between embedding-equivalent skills - that IS the failure mode being measured.

6.3 Terminal-Bench

- Single-judge GT (Opus 4.7). Two-judge agreement (Opus + GPT or Sonnet) would split into precision-leaning vs recall-leaning GT. Not yet done.
- Strict prompt may over-reject niche-but-valid skills; 8.1 % YES rate is likely a lower bound. The retrievability conditioning (next bullet) is the more important confound.
- DATASET CONSTRUCTION CONDITIONING: GT is selected from the top-5 retrievals per task, so all TB results are conditioned on GT being highly retrievable (rank ≤ 5 in the corpus). 33 / 62 tasks have `rank_GT = 1`; the remaining 29 are in [2, 5]; none deeper. Findings should be read as 'even under this favorable retrievability regime, the model fails to reliably select the correct skill under hard distractors' - which strengthens the claim, not weakens it.
- The 179 dropped tasks are characterised separately as a coverage-gap / refusal-calibration story (see RESULTS.md 2026-05-03 TB pilot section).

6.4 Methodology

- Ground truth is semantic (LLM-judged), not execution-validated. A delta-judge upgrade ('compare no-skill vs with-skill agent outputs') is the natural follow-up if Figure 1 is contested. Not currently planned because the embedding-argmax comparison already provides an execution-independent ceiling.

7. Open items

- Variance: re-run grids with seeds {0,1,2}.
- Multi-judge agreement on TB GT (precision-leaning subset).
- Cross-model eval on the validated grid.
- Multi-GT scoring on SB (drop `max_gt=1`).
- Position-bias control: shuffle `topk_only` ordering.
- Optional: delta-judge GT upgrade if reviewers contest semantic GT.

8. Files in the commit

Authoritative

data/skillsbench_tasks.jsonl	88	in-domain testset, native multi-GT
data/terminal_bench_tasks.jsonl	241	raw TB tasks (no GT)
data/terminal_bench_topk.jsonl	241	FAISS top-5 candidates per TB task
data/terminal_bench_validated.jsonl	62	OOD testset, Opus-judge multi-GT
data/terminal_bench_validated_summary.md		Aggregator summary

Code

skill_selection_eval/	
judge_tb.py	Opus per-(task, skill) YES/NO judge
aggregate_judge.py	judge JSONL -> validated.jsonl
retrieve.py / retrieve_topk.py / retrieve_topk_gpu.py	FAISS top-K + Qwen3-Embedding
embed_tasks.py / embed_gt.py	Task and GT embedding generation
pool_builder.py	SB pool factory (size, noise)
tb_pool_builder.py	TB pool factory (uses topk JSONL)
prompt.py	<available_skills> XML harbor-format
parser.py	<tool_call name="skill"> regex (harbor-identical)
run.py	SB run driver - Max-plan claude -p, resume-aware
run_tb.py	TB run driver - surfaces validated_gt_slugs
aggregate.py	SB metrics: Hit@1, Recall, MRR, refusal
aggregate_tb.py	TB metrics (proxy-GT compatible)
build_tb_tasks.py	Extract task descriptions from terminal-bench/ tree
position_probe.py	Position-bias sanity probes

Reports / figures

figures/	
collapse_analysis.pdf	Figure 1: rank CDF / delta-similarity / Hit@1 vs oracle
terminalbench-testset.pdf	TB construction + todo-completion summary
testsets-overview.pdf	This document - joint SB+TB report
RESULTS.md	Full per-section methodology + caveats
README.md	Quick-start usage guide